

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5**

“Процедуры, функции, триггеры в PostgreSQL”

Обучающийся Артемов Илья Витальевич

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Описание предметной области:

Вариант 9. БД «Оптовая база»

Описание предметной области: Оптовая база закупает товары у компаний-поставщиков и поставляет их компаниям – покупателям. Компании поставщики не являются производителями товара. Доход оптовой базы составляет не менее 5% от стоимости товара, проданного компании-покупателю. Каждый товар имеет производителя. Один и тот же товар может доставляться несколькими поставщиками, и один и тот же поставщик может поставлять несколько видов товаров. Цены поставки товара у разных поставщиков могут отличаться. В один заказ при покупке товара у оптовой базы может попасть товар от разных поставщиков, в зависимости от наличия на складе. Поставки и заказы обслуживают менеджеры по работе с клиентами (по поставкам и продажам).

ЧАСТЬ 1 — ПОДКЛЮЧЕНИЕ К PSQL

```
postgres=# \c wholesale_base
[You are now connected to database "wholesale_base" as user "postgres".
```

ЧАСТЬ 2 — СОЗДАНИЕ ТАБЛИЦЫ АУДИТА

Нужна для логирования в триггерах 5 и 7

```
wholesale_base=# CREATE TABLE IF NOT EXISTS wb_schema.audit_log (  
wholesale_base(#      log_id      SERIAL PRIMARY KEY,  
wholesale_base(#      table_name  VARCHAR(50),  
wholesale_base(#      operation   VARCHAR(10),  
wholesale_base(#      record_id   INTEGER,  
wholesale_base(#      changed_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
wholesale_base(#      changed_by  VARCHAR(50) DEFAULT CURRENT_USER,  
wholesale_base(#      description TEXT  
wholesale_base(# );  
CREATE TABLE
```

ЧАСТЬ 3 — ПРОЦЕДУРЫ

Процедура 1 — Снижение цены для залежавшихся товаров

Формулировка: Снизить продажную цену на заданный процент для товаров, у которых срок пребывания на складе превысил заданный норматив (дней).

Параметры:

- p_percent — процент снижения цены
- p_days — норматив хранения в днях

```
wholesale_base=# CREATE OR REPLACE PROCEDURE wb_schema.sp_discount_old_stock(  
wholesale_base(#   IN p_percent NUMERIC,  
wholesale_base(#   IN p_days   INTEGER  
wholesale_base(# )  
wholesale_base=# LANGUAGE plpgsql  
wholesale_base=# AS $$  
wholesale_base$# DECLARE  
wholesale_base$#     v_count INTEGER := 0;  
wholesale_base$# BEGIN  
wholesale_base$#     UPDATE wb_schema.order_item oi  
wholesale_base$#     SET unit_price = ROUND(oi.unit_price * (1 - p_percent / 100.0), 2)  
wholesale_base$#     WHERE oi.order_id IN (  
wholesale_base$#         SELECT order_id  
wholesale_base$#         FROM wb_schema."order"  
wholesale_base$#         WHERE pickup_date IS NULL  
wholesale_base$#     )  
wholesale_base$#     AND oi.product_id IN (  
wholesale_base$#         SELECT di.product_id  
wholesale_base$#         FROM wb_schema.delivery_item di  
wholesale_base$#         JOIN wb_schema.delivery d ON di.delivery_id = d.delivery_id  
wholesale_base$#         GROUP BY di.product_id  
wholesale_base$#         HAVING MIN(d.delivery_date) < CURRENT_DATE - (p_days || ' days')::INTERVAL  
wholesale_base$#     );  
wholesale_base$#     GET DIAGNOSTICS v_count = ROW_COUNT;  
wholesale_base$#     RAISE NOTICE 'Скидка %%% применена к % позициям (товары на складе более % дней)',  
wholesale_base$#         p_percent, v_count, p_days;  
wholesale_base$# END;  
wholesale_base$# $$;  
CREATE PROCEDURE =
```

Тест процедуры 1:

```
SELECT oi.order_item_id, p.name, oi.unit_price
```

```
FROM wb_schema.order_item oi
```

```
JOIN wb_schema.product p ON oi.product_id = p.product_id
```

```
JOIN wb_schema."order" o ON oi.order_id = o.order_id
```

```
WHERE o.pickup_date IS NULL;
```

order_item_id	name	unit_price
11	Кефир 2.5%	40.95
12	Говядина охл.	400.75
(2 rows)		

Вызываем процедуру: скидка 10% для товаров на складе более 2 дней

```
CALL wb_schema.sp_discount_old_stock(10, 2);
```

```
SELECT oi.order_item_id, p.name, oi.unit_price
```

```
FROM wb_schema.order_item oi
```

```
JOIN wb_schema.product p ON oi.product_id = p.product_id
```

```
JOIN wb_schema."order" o ON oi.order_id = o.order_id
```

```
WHERE o.pickup_date IS NULL;
```

order_item_id	name	unit_price
11	Кефир 2.5%	36.86
12	Говядина охл.	360.68
(2 rows)		

Процедура 2 — Расчёт стоимости продаж за прошедшие сутки

Формулировка: Рассчитать стоимость всех партий товаров, проданных за прошедшие сутки.

Параметр: p_total (INOUT) — возвращает итоговую сумму.

```
wholesale_base=# CREATE OR REPLACE PROCEDURE wb_schema.sp_calc_daily_revenue(  
wholesale_base(#      INOUT p_total NUMERIC  
wholesale_base(# )  
wholesale_base=# LANGUAGE plpgsql  
wholesale_base=# AS $$  
wholesale_base$$ BEGIN  
wholesale_base$$     SELECT COALESCE(SUM(oi.quantity * oi.unit_price), 0)  
wholesale_base$$     INTO p_total  
wholesale_base$$     FROM wb_schema.order_item oi  
wholesale_base$$     JOIN wb_schema."order" o ON oi.order_id = o.order_id  
wholesale_base$$     WHERE o.order_date = CURRENT_DATE - INTERVAL '1 day';  
wholesale_base$$     RAISE NOTICE 'Выручка за %: %', CURRENT_DATE - INTERVAL '1 day', p_total;  
wholesale_base$$ END;  
wholesale_base$$ $$;  
CREATE PROCEDURE
```

Тест процедуры 2:

DO \$\$

DECLARE

 v_total NUMERIC := 0;

BEGIN

 CALL wb_schema.sp_calc_daily_revenue(v_total);

 RAISE NOTICE 'Итоговая выручка: %', v_total;

END;

\$\$;

```
NOTICE:  Выручка за 2026-05-29 00:00:00: 5145.20  
NOTICE:  Итоговая выручка: 5145.20  
DO
```

Процедура 3 — Регистрация нового заказа

Формулировка: Зарегистрировать новый заказ для указанного покупателя и менеджера, вернуть номер созданного заказа.

Параметры:

- p_org_id (IN) — код организации-покупателя
- p_emp_id (IN) — табельный номер менеджера
- p_order_id (INOUT) — возвращает ID созданного заказа

```
wholesale_base=# CREATE OR REPLACE PROCEDURE wb_schema.sp_register_order(
wholesale_base(#      IN      p_org_id  INTEGER,
wholesale_base(#      IN      p_emp_id  INTEGER,
wholesale_base(#      INOUT  p_order_id INTEGER
wholesale_base(# )
wholesale_base=# LANGUAGE plpgsql
wholesale_base=# AS $$
wholesale_base$# BEGIN
wholesale_base$#     INSERT INTO wb_schema."order"
wholesale_base$#         (organization_id, personnel_number, order_date, pickup_date)
wholesale_base$#         VALUES
wholesale_base$#             (p_org_id, p_emp_id, CURRENT_DATE, NULL)
wholesale_base$#     RETURNING order_id INTO p_order_id;
wholesale_base$#
wholesale_base$#     RAISE NOTICE 'Создан новый заказ № % для организации %', p_order_id, p_org_id;
wholesale_base$# END;
wholesale_base=# $$;
CREATE PROCEDURE
```

Тест процедуры 3:

```
SELECT order_id, organization_id, order_date
```

```
FROM wb_schema."order"
```

```
ORDER BY order_id DESC LIMIT 3;
```

order_id	organization_id	order_date
8	2	2025-03-29
7	1	2025-04-29
6	1	2026-05-19

(3 rows)

```
DO $$
```

```
DECLARE
```

```

v_new_id INTEGER := 0;

BEGIN

    CALL wb_schema.sp_register_order(2, 4, v_new_id);

    RAISE NOTICE 'Новый заказ получил ID: %', v_new_id;

END;

$$;

```

```

NOTICE:  Создан новый заказ № 10 для организации 2
NOTICE:  Новый заказ получил ID: 10
DO

```

```

SELECT order_id, organization_id, order_date
FROM wb_schema."order"
ORDER BY order_id DESC LIMIT 3;

```

order_id	organization_id	order_date
10	2	2026-05-30
8	2	2025-03-29
7	1	2025-04-29

(3 rows)

ЧАСТЬ 4 — ТРИГГЕРЫ

Триггер 1 — Контроль минимальной цены продажи (INSERT)

Формулировка: При добавлении позиции в заказ проверять, что продажная цена не ниже закупочной более чем на 5%.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_check_min_price()
wholesale_base=# RETURNS trigger AS $$
wholesale_base=# DECLARE
wholesale_base=#     v_avg_cost NUMERIC;
wholesale_base=# BEGIN
wholesale_base=#     SELECT AVG(unit_cost) INTO v_avg_cost
wholesale_base=#     FROM wb_schema.delivery_item
wholesale_base=#     WHERE product_id = NEW.product_id;
wholesale_base=#     IF v_avg_cost IS NOT NULL AND NEW.unit_price < ROUND(v_avg_cost * 1.05, 2) THEN
wholesale_base=#         RAISE EXCEPTION
wholesale_base=#         'Цена продажи % ниже минимально допустимой % (закупка + 5%)',
wholesale_base=#         NEW.unit_price, ROUND(v_avg_cost * 1.05, 2);
wholesale_base=#     END IF;
wholesale_base=#     RETURN NEW;
wholesale_base=# END;
wholesale_base=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_check_min_price_insert
wholesale_base=# BEFORE INSERT ON wb_schema.order_item
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_check_min_price();
CREATE TRIGGER
```

Тест триггера 1:

```
wholesale_base=# INSERT INTO wb_schema.order_item (order_id, product_id, quantity, unit_price)
wholesale_base=# VALUES (1, 6, 5, 57.75);
INSERT 0 1
```

```
wholesale_base=# INSERT INTO wb_schema.order_item (order_id, product_id, quantity, unit_price)
wholesale_base=# VALUES (1, 6, 5, 10.00);
ERROR:  Цена продажи 10.00 ниже минимально допустимой 57.75 (закупка + 5%)
CONTEXT:  PL/pgSQL function fn_check_min_price() line 10 at RAISE
```

Триггер 2 — Запрет смены покупателя в существующем заказе (UPDATE)

Формулировка: Запретить изменение организации-покупателя в уже созданном заказе.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_prevent_org_change()
wholesale_base=# RETURNS trigger AS $$
wholesale_base$# BEGIN
wholesale_base$#     IF OLD.organization_id <> NEW.organization_id THEN
wholesale_base$#         RAISE EXCEPTION
wholesale_base$#             'Нельзя изменить покупателя для заказа №%. Текущий: %, Новый: %',
wholesale_base$#             OLD.order_id, OLD.organization_id, NEW.organization_id;
wholesale_base$#     END IF;
wholesale_base$#     RETURN NEW;
wholesale_base$# END;
wholesale_base$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_prevent_org_change
wholesale_base=# BEFORE UPDATE ON wb_schema."order"
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_prevent_org_change();
CREATE TRIGGER
```

Тест триггера 2:

```
wholesale_base=# UPDATE wb_schema."order"
wholesale_base=# SET pickup_date = CURRENT_DATE
wholesale_base=# WHERE order_id = 1;
UPDATE 1
```

```
wholesale_base=# UPDATE wb_schema."order"
wholesale_base=# SET organization_id = 3
wholesale_base=# WHERE order_id = 1;
ERROR:  Нельзя изменить покупателя для заказа №1. Текущий: 1, Новый: 3
CONTEXT:  PL/pgSQL function fn_prevent_org_change() line 4 at RAISE
```

Триггер 3 — Проверка наличия товара на складе (INSERT)

Формулировка: При добавлении товара в заказ проверять достаточность остатка на складе.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_check_stock()
wholesale_base=# RETURNS trigger AS $$
wholesale_base=# DECLARE
wholesale_base$#     v_delivered INTEGER;
wholesale_base$#     v_sold      INTEGER;
wholesale_base$#     v_stock     INTEGER;
wholesale_base$# BEGIN
wholesale_base$#     SELECT COALESCE(SUM(quantity), 0) INTO v_delivered
wholesale_base$#     FROM wb_schema.delivery_item
wholesale_base$#     WHERE product_id = NEW.product_id;
wholesale_base$#
wholesale_base$#     SELECT COALESCE(SUM(quantity), 0) INTO v_sold
wholesale_base$#     FROM wb_schema.order_item
wholesale_base$#     WHERE product_id = NEW.product_id;
wholesale_base$#
wholesale_base$#     v_stock := v_delivered - v_sold;
wholesale_base$#
wholesale_base$#     IF v_stock < NEW.quantity THEN
wholesale_base$#         RAISE EXCEPTION
wholesale_base$#             'Недостаточно товара (product_id=%). На складе: %, Запрошено: %',
wholesale_base$#             NEW.product_id, v_stock, NEW.quantity;
wholesale_base$#     END IF;
wholesale_base$#     RETURN NEW;
wholesale_base$# END;
wholesale_base$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_check_stock
wholesale_base=# BEFORE INSERT ON wb_schema.order_item
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_check_stock();
CREATE TRIGGER
```

Тест триггера 3:

```
wholesale_base=# INSERT INTO wb_schema.order_item (order_id, product_id, quantity, unit_price)
wholesale_base=# VALUES (1, 6, 5, 57.75);
INSERT 0 1
```

```
wholesale_base=# INSERT INTO wb_schema.order_item (order_id, product_id, quantity, unit_price)
wholesale_base=# VALUES (1, 6, 99999, 57.75);
ERROR:  Недостаточно товара (product_id=6). На складе: 230, Запрошено: 99999
CONTEXT:  PL/pgSQL function fn_check_stock() line 18 at RAISE
```

Триггер 4 — Контроль корректности данных поставки (INSERT/UPDATE)

Формулировка: При добавлении или изменении позиции поставки проверять, что количество и цена положительные.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_check_delivery_values()
wholesale_base=# RETURNS trigger AS $$
wholesale_base=# BEGIN
wholesale_base=#     IF NEW.quantity <= 0 THEN
wholesale_base=#         RAISE EXCEPTION
wholesale_base=#         'Количество в поставке должно быть больше 0, получено: %', NEW.quantity;
wholesale_base=#     END IF;
wholesale_base=#
wholesale_base=#     IF NEW.unit_cost <= 0 THEN
wholesale_base=#         RAISE EXCEPTION
wholesale_base=#         'Закупочная цена должна быть больше 0, получено: %', NEW.unit_cost;
wholesale_base=#     END IF;
wholesale_base=#     RETURN NEW;
wholesale_base=# END;
wholesale_base=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_check_delivery_values
wholesale_base=# BEFORE INSERT OR UPDATE ON wb_schema.delivery_item
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_check_delivery_values();
CREATE TRIGGER
```

Тест триггера 4:

```
wholesale_base=# INSERT INTO wb_schema.delivery_item (delivery_id, product_id, quantity, unit_cost)
wholesale_base=# VALUES (1, 6, 50, 55.00);
INSERT 0 1
```

```
wholesale_base=# INSERT INTO wb_schema.delivery_item (delivery_id, product_id, quantity, unit_cost)
wholesale_base=# VALUES (1, 6, -10, 55.00);
ERROR:  Количество в поставке должно быть больше 0, получено: -10
CONTEXT:  PL/pgSQL function fn_check_delivery_values() line 4 at RAISE
```

Триггер 5 — Логирование новых заказов (AFTER INSERT)

Формулировка: При создании нового заказа автоматически записывать событие в журнал аудита.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_log_new_order()
wholesale_base=# RETURNS trigger AS $$
wholesale_base$$ BEGIN
wholesale_base$$     INSERT INTO wb_schema.audit_log
wholesale_base$$         (table_name, operation, record_id, description)
wholesale_base$$     VALUES (
wholesale_base$$         'order',
wholesale_base$$         'INSERT',
wholesale_base$$         NEW.order_id,
wholesale_base$$         'Создан заказ №' || NEW.order_id ||
wholesale_base$$         ', организация: ' || NEW.organization_id ||
wholesale_base$$         ', дата: ' || NEW.order_date
wholesale_base$$     );
wholesale_base$$     RETURN NEW;
wholesale_base$$ END;
wholesale_base$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_log_new_order
wholesale_base=# AFTER INSERT ON wb_schema."order"
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_log_new_order();
CREATE TRIGGER
```

Тест триггера 5:

```
wholesale_base=# INSERT INTO wb_schema."order" (organization_id, personnel_number, order_date, pickup_date)
wholesale_base=# VALUES (3, 3, CURRENT_DATE, NULL);
INSERT 0 1
```

```
wholesale_base=# SELECT * FROM wb_schema.audit_log ORDER BY log_id DESC LIMIT 3;
 log_id | table_name | operation | record_id |      changed_at      | changed_by | description
-----+-----+-----+-----+-----+-----+-----
      1 | order      | INSERT   |         11 | 2026-05-30 09:49:45.227367 | postgres   | Создан заказ №11, организация: 3, дата: 2026-05-30
(1 row)
```

Триггер 6 — Запрет удаления поставки с позициями (BEFORE DELETE)

Формулировка: Запретить удаление поставки, если в ней есть товарные позиции.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_prevent_delivery_delete()
wholesale_base=# RETURNS trigger AS $$
wholesale_base$# DECLARE
wholesale_base$#     v_count INTEGER;
wholesale_base$# BEGIN
wholesale_base$#     SELECT COUNT(*) INTO v_count
wholesale_base$#     FROM wb_schema.delivery_item
wholesale_base$#     WHERE delivery_id = OLD.delivery_id;
wholesale_base$#
wholesale_base$#     IF v_count > 0 THEN
wholesale_base$#         RAISE EXCEPTION
wholesale_base$#         'Нельзя удалить поставку №% — содержит % позиций. Сначала удалите позиции.',
wholesale_base$#         OLD.delivery_id, v_count;
wholesale_base$#     END IF;
wholesale_base$#
wholesale_base$#     RETURN OLD;
wholesale_base$# END;
wholesale_base$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_prevent_delivery_delete
wholesale_base=# BEFORE DELETE ON wb_schema.delivery
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_prevent_delivery_delete();
CREATE TRIGGER
```

Тест триггера 6:

```
wholesale_base=# DELETE FROM wb_schema.delivery WHERE delivery_id = 1;
ERROR:  Нельзя удалить поставку №1 — содержит 7 позиций. Сначала удалите позиции.
CONTEXT:  PL/pgSQL function fn_prevent_delivery_delete() line 10 at RAISE
```

Триггер 7 — Логирование изменения закупочной цены (AFTER UPDATE)

Формулировка: При изменении закупочной цены товара в поставке автоматически фиксировать изменение в журнале аудита.

```
wholesale_base=# CREATE OR REPLACE FUNCTION wb_schema.fn_log_price_change()
wholesale_base=# RETURNS trigger AS $$
wholesale_base$# BEGIN
wholesale_base$#     IF OLD.unit_cost <> NEW.unit_cost THEN
wholesale_base$#         INSERT INTO wb_schema.audit_log
wholesale_base$#             (table_name, operation, record_id, description)
wholesale_base$#             VALUES (
wholesale_base$#                 'delivery_item',
wholesale_base$#                 'UPDATE',
wholesale_base$#                 NEW.delivery_item_id,
wholesale_base$#                 'Изменена цена товара ' || NEW.product_id ||
wholesale_base$#                 ': ' || OLD.unit_cost || ' -> ' || NEW.unit_cost
wholesale_base$#             );
wholesale_base$#     END IF;
wholesale_base$#     RETURN NEW;
wholesale_base$# END;
wholesale_base$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
wholesale_base=# CREATE TRIGGER trg_log_price_change
wholesale_base=# AFTER UPDATE ON wb_schema.delivery_item
wholesale_base=# FOR EACH ROW EXECUTE FUNCTION wb_schema.fn_log_price_change();
CREATE TRIGGER
```

Тест триггера 7:

```
wholesale_base=# UPDATE wb_schema.delivery_item
wholesale_base=# SET unit_cost = 46.00
wholesale_base=# WHERE delivery_item_id = 1;
UPDATE 1
```

```
wholesale_base=# SELECT * FROM wb_schema.audit_log ORDER BY log_id DESC LIMIT 3;
```

log_id	table_name	operation	record_id	changed_at	changed_by	description
2	delivery_item	UPDATE	1	2026-05-30 09:52:40.439908	postgres	Изменена цена товара 1: 45.00 -> 46.00
1	order	INSERT	11	2026-05-30 09:49:45.227367	postgres	Создан заказ №11, организация: 3, дата: 2026-05-30

(2 rows)

Выводы

В ходе лабораторной работы были получены практические навыки создания и использования хранимых процедур и триггеров в СУБД PostgreSQL с использованием консольного клиента psql.

Были разработаны три хранимые процедуры для базы данных «Оптовая база»: процедура снижения цены на залежавшиеся товары, процедура расчёта суточной выручки с использованием параметра INOUT, и процедура регистрации нового заказа. Использование процедур позволяет инкапсулировать бизнес-логику на стороне сервера, сократить объём передаваемых данных и обеспечить повторное использование кода.

Также были созданы 7 триггеров, охватывающих операции INSERT, UPDATE и DELETE на различных таблицах. Триггеры выполняют две основные функции: валидацию данных (контроль минимальной цены продажи, проверка наличия товара на складе, проверка корректности данных поставки, защита от некорректных изменений) и аудит действий (автоматическое логирование новых заказов и изменений закупочных цен в таблицу журнала). Это обеспечивает целостность данных и прозрачность работы с базой без участия приложения.